

Table of contents:

-  [Introducción](#)
-  [Despliegue con GitHub Pages](#)
-  [Internacionalización \(i18n\)](#)
-  [Buscador Integrado](#)
-  [Versionado de Apuntes](#)
-  [Exportación Offline a PDF](#)

Introducción

- [Objetivos de la sesión](#)

Despliegue con GitHub Pages

- [Configuración de Docusaurus](#)
- [Entorno de Repositorio \(Paso Único\)](#)
- [Operación de Despliegue \(Publish\)](#)
 - [Analizando el comando de despliegue](#)
- [Flujo Normal de Trabajo \(Con Despliegue\)](#)

Internacionalización (i18n)

- [Configuración de Idiomas](#)
- [Organizar las Carpetas de Traducción](#)
- [Ver el Resultado en Local](#)

Buscador Integrado

- [Integración con Algolia paso a paso](#)
 - [1. Registrarse y solicitar el rastreo](#)
 - [2. Configurar Docusaurus](#)
 - [3. Prueba el Buscador en Local](#)

Versionado de Apuntes

- [Estrategia 1: Versionado Manual por Carpetas](#)
- [Estrategia 2: Versionado Automático Global](#)
 - [Cómo congelar tus apuntes actuales](#)
 - [Añade el menú temporal](#)
 - [Evitar el cartel de "Versión no publicada"](#)

Exportación Offline a PDF

- [Ejecutar el generador automático \(docs-to-pdf\)](#)
 - [¿Dónde encuentro el archivo?](#)



Introducción

Introducción a la última sesión sobre lanzamiento a producción, internacionalización y escalabilidad de la document...



Despliegue con GitHub Pages

Publicación del curso en producción de forma manual usando el compilador de Docusaurus hacia GitHub Pages.



Internacionalización (i18n)

Configuración de Docusaurus para soportar múltiples idiomas en los apuntes.



Buscador Integrado

Guía paso a paso para añadir un motor de rastreo y sugerencias a tus apuntes utilizando Algolia DocSearch.



Versionado de Apuntes

Estrategia para retener el histórico de apuntes de diferentes cursos académicos.



Exportación Offline a PDF

Generación automatizada de apuntes en formato PDF para los alumnos.

Introducción

Bienvenidos a la tercera y última sesión del curso sobre Docusaurus. En esta etapa final cerraremos el ciclo completo de *Docs-as-Code* enfocándonos en el lanzamiento a producción de nuestros apuntes y garantizando su escalabilidad docente para los próximos cursos.

Objetivos de la sesión

Durante esta sesión abordaremos las siguientes competencias técnicas y metodológicas:

- **Despliegue Automatizado:** Configuración de flujos de trabajo (*workflows*) en GitHub Actions para publicar el temario automáticamente en GitHub Pages de manera pública y gratuita.
- **Soporte Multilingüe (i18n):** Adaptación de la arquitectura web para albergar diferentes idiomas (por ejemplo, castellano e inglés o lenguas cooficiales).
- **Motores de Búsqueda para Producción:** Integración del potente escáner oficial Algolia DocSearch.
- **Versionado de Temario:** Estrategia nativa de Docusaurus para congelar materiales de cursos académicos pasados, posibilitando el repaso del alumnado repetidor sin comprometer la limpieza del curso entrante.
- **Exportación Offline:** Herramientas que permiten volcar cada sesión directamente a formato PDF de forma autónoma.

CONSEJO DE SESIÓN

Al tratarse de modificaciones profundas a nivel de arquitectura, procesos e inyección de plugins principales, es altamente probable que necesites pausar (`Ctrl + C`) y reiniciar asiduamente el servidor de desarrollo local (`npm run start`). Si no detectas reflejados tus cambios, fuérmalo de este modo.

Despliegue con GitHub Pages

En este apartado documentaremos el proceso para publicar nuestros apuntes en internet a voluntad. Aunque la automatización total es potente, en el ecosistema educativo suele ser preferible conservar el control manual sobre **cuándo** se publican los cambios a producción. De ese modo, podemos guardar nuestro trabajo de resúmenes o borradores en GitHub sin peligro de que los alumnos lo lean prematuramente.

Por ello, utilizaremos el comando nativo de despliegue de Docusaurus: compilará y publicará la web en **GitHub Pages** únicamente en el momento que nosotros lo ordenemos explícitamente.

Configuración de Docusaurus

Antes de poder realizar el despliegue, debemos ajustar ciertos parámetros en el núcleo de Docusaurus para que entienda en qué dominio y ruta vivirá el sitio al estar público. Abre tu `docusaurus.config.js` y edita las siguientes variables en la raíz de tu configuración:

docusaurus.config.js

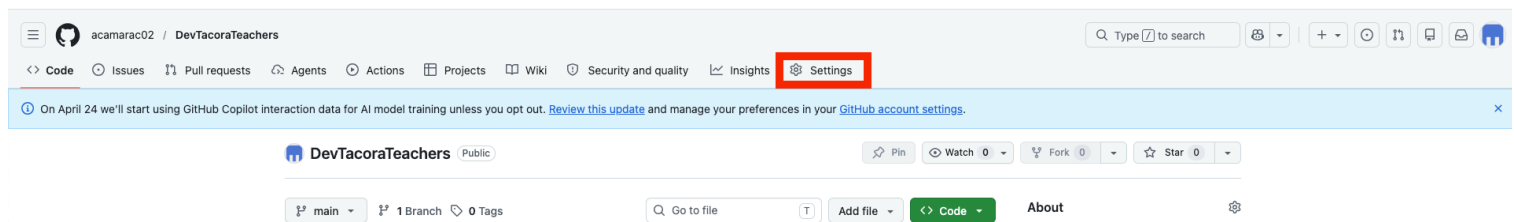
```
const config = {  
  // Dominios y rutas de producción  
  url: 'https://TU-USUARIO.github.io', // Cambia TU-USUARIO por tu nombre en GitHub  
  baseUrl: '/NOMBRE-DEL-REPO/', // Nombre exacto de tu repositorio. Observa las  
    barras "/"  
  
  // Metadatos requeridos por el script de despliegue  
  organizationName: 'TU-USUARIO', // Usualmente tu nombre de usuario de GitHub  
  projectName: 'NOMBRE-DEL-REPO', // Nombre del repositorio  
  deploymentBranch: 'gh-pages', // Rama oculta que creará Docusaurus para alojar la  
web  
  trailingSlash: false,  
  
  // ...resto de opciones de Docusaurus...
```

Al alojar este proyecto en tu propia cuenta formativa de GitHub, la URL resultante no es un dominio absoluto sino que cuelga sobre tu nombre (`.io/repo/`). Por ello es mandatorio preservar la propiedad `baseUrl` rodeada de barras invertidas (`/`).

Entorno de Repositorio (Paso Único)

Debemos configurar nuestro repositorio en GitHub para indicarle que el origen visual de la página web será la rama donde Docusaurus volcará el resultado. Esta acción de mantenimiento solo recae en nosotros la primera vez:

1. Ingresa a la página principal de tu repositorio específico en [GitHub.com](https://github.com) (por ejemplo: `github.com/tu-usuario/tu-repo`).
2. Dirígete a la pestaña **Settings** (Ajustes) que aparece en el menú horizontal del propio repositorio (junto a *Code*, *Issues*, *Pull requests*, etc.). **Importante:** No debes pulsar en los *Settings* globales de tu perfil de usuario situados arriba a la derecha.



3. Tras entrar, en el menú lateral izquierdo, haz clic en **Pages**.
4. En la sección **Build and deployment**, busca el menú desplegable asignado al parámetro **Source** y asegúrate de elegir la opción clásica **Deploy from a branch**.
5. Justo debajo, en el bloque **Branch**, selecciona la rama que hemos bautizado como `gh-pages` y pulsa **Save**.

(Nota: Si GitHub no te permite seleccionar la rama `gh-pages` porque afirma que "no existe", significa simplemente que debes esperar a lanzar el comando de despliegue de la siguiente sección por primera vez para que Docusaurus la genere automática).

Operación de Despliegue (Publish)

Cuando certifiques que tus apuntes están listos para ser consumidos por el aula y quieras publicarlos, el proceso consiste exclusivamente en ejecutar la orden de publicación local.

⚠ PRIMER DESPLIEGUE

Si es la **primera vez** que vas a publicar el sitio, deberás crear la rama `gh-pages` en GitHub. Para ello, escribe este comando por única vez en la consola de tu editor:

```
git push origin HEAD:gh-pages
```

Una vez empujado el esqueleto básico en GitHub, el flujo normal ya consiste en ejecutar el comando de despliegue.

Abre la terminal integrada en tu editor y lanza el proceso estandarizado de compilación. Por defecto, Docusaurus detectará inteligentemente las credenciales de GitHub que ya usamos a diario en nuestro flujo de trabajo:

```
npm run deploy
```

(Nota de seguridad: Si en un futuro cambias de ordenador o la consola informa de problemas de permisos por tener varias cuentas de Github cruzadas, puedes forzar empíricamente el usuario prefijando la variable de esta forma: `GIT_USER=TU-USUARIO npm run deploy`).

Analizando el comando de despliegue

En cuestión de un minuto, esta instrucción unificada realizará el siguiente listado de tareas invisibles en tu máquina local:

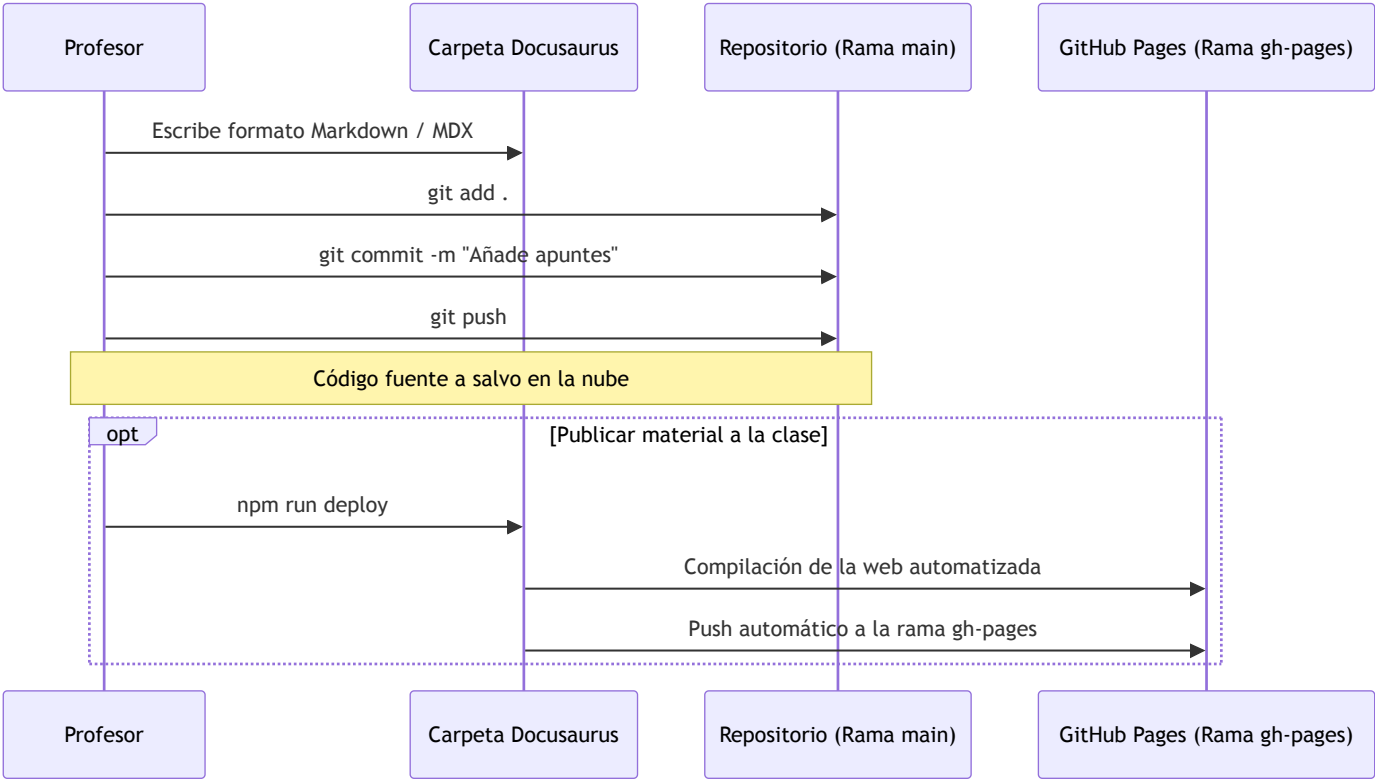
1. Construirá y empaquetará una versión ultraligera y cacheable de todos tus archivos Markdown y componentes React (exactamente igual que usar `npm run build`).
2. Traspasará ese resultado minificado en la rama aislada `gh-pages`.
3. Hará el empuje (*push*) automático de esa pequeña rama sobre el repositorio remoto público de GitHub, desencadenando la exposición inmediata del temario web.
4. Garantizará que la rama esencial de trabajo (`main`) se mantenga inmaculada como simple código fuente, a salvo de basura de compilación.

Flujo Normal de Trabajo (Con Despliegue)

Para tu día a día como docente, el ciclo de trabajo de los apuntes queda resumido y estandarizado en el siguiente proceso lógico cada vez que prepares una nueva clase:

- 1. Modificas o creas los archivos `.md`.
- 2. Haces seguimiento de los cambios en Git: `git add .`
- 3. Confirmas el hito de tu nuevo trabajo: `git commit -m "Terminada la unidad 4"`
- 4. Subes tu código fuente original al repositorio como salvaguarda: `git push`
- 5. Si además evalúas que este material ya debe ser visible en internet por tus alumnos, ejecutas el empaquetado: `npm run deploy`.

Visualmente de forma arquitectónica, el flujo de interacciones se diagrama de este modo:



Internacionalización (i18n)

En muchos centros de Formación Profesional existen ciclos bilingües, o bien necesitamos tener los temarios disponibles en una segunda lengua oficial. Docusaurus trae un sistema integrado de idiomas (i18n) que nos facilita muchísimo este trabajo, sin tener que duplicar nuestro proyecto web entero.

En este tutorial vamos a ver cómo preparar un segundo idioma (usaremos el **inglés**, bajo el código `en`) partiendo de nuestra web principal actual (el **castellano**, con el código `es`).

Configuración de Idiomas

Lo primero es decirle a nuestra página web qué idiomas vamos a manejar. Para ello, debemos editar el documento principal `docusaurus.config.js`. Busca el bloque `i18n` que suele estar arriba del todo:

`docusaurus.config.js`

```
export default {
  // ...
  i18n: {
    defaultLocale: 'es',           // El lenguaje principal de tu página (tu carpeta
docs/ original)
    locales: ['es', 'en'],         // Lista con todos los lenguajes soportados en la web
    localeConfigs: {
      es: {
        label: 'Castellano',
        direction: 'ltr',
      },
      en: {
        label: 'English',
        direction: 'ltr',
      },
    },
  },
  // ...
};
```

Luego necesitamos indicarle al menú superior que muestre el botón o la pestaña de cambiar de idioma. Baja en ese mismo documento hasta la sección `themeConfig > navbar > items` y añade el botón

localeDropdown:

docusaurus.config.js

```
items: [  
  // ... tus otros menús ...  
  {  
    type: 'localeDropdown',  
    position: 'right', // Posiciona el selector en la esquina derecha  
  },  
],
```

Con esto, los alumnos verán un desplegable visual cómodo en el menú de la web para alternar entre apuntes con un solo clic.

Organizar las Carpetas de Traducción

Docusaurus no traduce el contenido automáticamente. Su filosofía se basa en que creamos una replica exacta de las carpetas donde guardamos nuestros archivos `.md` y ahí vayamos traduciéndolos manualmente para conservar el control total.

Para que reconozca los nuevos idiomas, hay que seguir esta ruta de trabajo en tu código:

1. Crea una carpeta padre llamada `i18n/` en la raíz de tu proyecto (junto a tus antiguas carpetas `docs/` y `src/`).
2. Dentro de la nueva carpeta, crea la estructura obligatoria de Docusaurus para el idioma inglés terminando con los documentos: `i18n/en/docusaurus-plugin-content-docs/current/`.
3. Copia ahí los archivos `.md` de temas que quieras traducir, exactamente con la misma jerarquía de subcarpetas que tenían originalmente.
4. **La regla de oro:** Traduce **solo y exclusivamente** los textos de dentro del archivo `.md`. Nunca cambies el nombre del propio archivo físico ni de su carpeta, ya que si los nombres son distintos, los enlaces se romperán en el cambio de idioma.

Ejemplo visual con unos apuntes del módulo de PMDM:

```
raíz-del-proyecto/  
├── docs/  
│   └── 01-pmdm/  
│       └── 01-introduccion-android.md    <-- (Apunte original escrito en Castellano)
```

```
|
|
├─ i18n/
│   └─ en/
│       └─ docusaurus-plugin-content-docs/
│           └─ current/
│               └─ 01-pmdm/
│                   └─ 01-introduccion-android.md  <-- (Archivo clonado donde
escribiremos en Inglés)
└─ docusaurus.config.js
```

❗ ¿DÓNDE DEBEN IR TUS ORIGINALES EN CASTELLANO?

Puesto que hemos decidido que nuestro idioma base (por defecto) sea `es`, todos los apuntes iniciales en español no debes moverlos, tienen que seguir viviendo en la carpeta `docs/`. Dentro de la carpeta `i18n/` **solo** meterás las versiones en inglés u otros idiomas añadidos.

💡 TRADUCIR BOTONES DE LA PANTALLA

Si quieres traducir detalles propios del programa (como los mensajes de "Pasar a página siguiente", "Siguiente Tema" etc), puedes ejecutar el comando `npm run write-translations -- --locale en` en la terminal. Éste creará archivos con textos sueltos en tu carpeta de inglés listos para que los modifiques a mano.

Ver el Resultado en Local

Si estás trabajando en tu ordenador y ejecutas tu comando habitual `npm run start`, comprobarás un detalle llamativo: **el desplegable de idiomas dará un error de "Página no encontrada" (404) si intentas pulsarlo.**

Tranquilidad, esto es el comportamiento normal y no has roto nada. Por motivos de rendimiento, el servidor local de pruebas solo es capaz de renderizar y mostrar **un idioma a la vez**. Solo cuando publiques tus apuntes definitivamente usando el comando de despliegue (`npm run deploy`) el botón funcionará perfectamente al trenzar ambas vistas.

Por lo tanto, si quieres auditar a nivel local cómo ha quedado tu temario escrito en inglés, tienes que detener la consola (`Ctrl+C`) y forzar su arranque explícitamente en modo bilingüe lanzando esta orden en tu terminal:

```
npm run start -- --locale en
```

Con este comando, NodeJS asume que debe obviar de momento tus archivos originales de `/docs` y pasa a pintar la pantalla leyendo exclusivamente los apuntes de tu carpeta `/i18n/en`.

Buscador Integrado

Cuando nuestro temario empieza a crecer, añadir un buscador integrado se vuelve casi obligatorio. Ayudaremos a los estudiantes a localizar conceptos rápidamente para repasar, igual que hacen los desarrolladores en la documentación técnica profesional.

En este manual vamos a configurar **Algolia DocSearch**, que es el motor de búsqueda gratuito que los propios creadores de Docusaurus recomiendan y traen preparado por defecto.

Integración con Algolia paso a paso

Algolia utiliza un rastreador en sus servidores que lee automáticamente tu página web cuando la publicas y construye un índice. Esto permite ofrecer autocompletado y sugerencias casi instantáneas, siendo totalmente gratis y sin cargar el rendimiento de nuestra página web.

1. Registrarse y solicitar el rastreo

Como es un servicio gratuito reservado exclusivamente para páginas de documentación abierta, tenemos que enviarles la dirección web de nuestros apuntes para que verifiquen que somos un proyecto educativo:

1. Entra en la web de [Algolia DocSearch](#) y regístrate.
2. Una vez dentro, accederás a un sencillo asistente de creación (*wizard*) donde deberás rellenar **la dirección web final (URL)** donde ya tienes desplegados tus apuntes. Por ejemplo, en el caso de este curso sería `https://acamarac02.github.io/DevTacoraTeachers/`.

 **Get Started** |  **DOCSEARCH** Skip for now >

1 Add Domain

2 Crawl website

3 Implement DocSearch

Add domain for verification

Provide your documentation website or technical blog domain that contains the data you want to search. You'll be able to add more domains or subdomains after finishing onboarding. Note: all domains require domain ownership verification.

Domain

Add domain

Use an exact domain match. Wildcards (e.g. *.algolia.com) are not supported.

3. Tras enviar el formulario, el equipo de Algolia revisará tu web (suele tardar menos de 48 horas) y te enviará un correo electrónico de confirmación. En ese email te facilitarán tres claves vitales que necesitamos para conectar el buscador: `appId`, `apiKey` e `indexName`.

2. Configurar Docusaurus

Abre tu archivo `docusaurus.config.js` y busca el gran bloque llamado `themeConfig`.

Solo tenemos que añadir una nueva sección llamada `algolia` dentro de ese bloque y rellenarla con los datos del mail. Al hacer esto, Docusaurus "pintará" la barra de búsqueda sola:

`docusaurus.config.js`

```
const config = {
  // ...
  themeConfig: {
    algolia: {
      // 1. Sustituye estos textos por las tres claves que te mandaron al e-mail
      appId: 'IDENTIFICADOR_1X',
      apiKey: 'APIKEY_PUBLICA_BUSQUEDA',
      indexName: 'NOMBRE_CLAVE_MODULO',

      // 2. Fundamental dejar esto en true si tienes también la web en inglés
      contextualSearch: true,

      // 3. Recomendado – Filtros extra (déjalo vacío si no lo controlas)
      searchParameters: {},
    },
    navbar: {
      title: 'DevTacora Teachers',
      // ...
    }
  }
}
```

3. Prueba el Buscador en Local

¡AVISO! REINICIA TU SERVIDOR

Cambios profundos como el buscador o colores principales necesitan que reinicies el entorno. En la terminal donde estés previsualizando el proyecto, pulsa `Ctrl+C` para detenerlo y vuelve a escribir `npm run start`.

Si no hay fallos al arrancar, verás que ha aparecido un botón de "Buscar" en la parte derecha del menú de navegación de tu web.

A partir de ahora, tanto tú como los alumnos podréis hacer clic en ella, o mejor aún, usar el atajo universal de teclado `Ctrl + K` (o `Cmd + K` en Mac) para encontrar cualquier palabra o apartado perdido del temario de forma inmediata.

Versionado de Apuntes

Llega el mes de junio, se acaba el curso académico y toca pensar en el siguiente. Muchos profesores borran cosas de Moodle, esconden temas o sobrescriben los apuntes para empezar a preparar el material del año que entra. ¿El problema? Si un alumno repetidor necesita consultar el temario exactamente como se impartió en su curso, ya no puede.

Para solucionar este salto generacional de temarios, proponemos dos grandes estrategias posibles usando Docusaurus: el método puramente artesanal por carpetas, o el método automatizado nativo del sistema.

Estrategia 1: Versionado Manual por Carpetas

Esta es la táctica más sencilla y natural si manejamos varias asignaturas a la vez y nos gusta dosificar la información a ritmo de clase.

Consiste en usar el nombre de nuestras carpetas principales para mantener cada año académico visualmente separado, ocultando carpetas pasadas apoyándonos en el gestor de ficheros. Por ejemplo, supongamos que impartías PMDM en la carpeta `2526-pmdm/`. Cuando llegue septiembre del próximo curso harás lo siguiente:

1. **Ocultar lo viejo:** Modificas el nombre de tu vieja carpeta `2526-pmdm/` para que Docusaurus la ignore, simplemente añadiéndole una barra baja delante: `_2526-pmdm/`. Así desaparece de la vista pública de los nuevos alumnos (pero ojo, **¡con este método los repetidores pierden su histórico!**).
2. **Crear lo nuevo:** Te creas una carpeta vacía llamada `2627-pmdm/`.
3. **Migración progresiva:** Según vayan pasando las semanas y vayas impartiendo los temas en clase, abres tu carpeta antigua oculta (`_2526-pmdm/`) y vas copiando poco a poco los archivos hacia tu nueva versión visible `2627-pmdm/`, aprovechando para reescribirlos o mejorarlos para la nueva promoción antes de que los vean.

Esta forma de trabajar manual te da un control asombroso y evita de raíz que los alumnos de primero vean de golpe los temas avanzados que vas a modificar más adelante en el trimestre.

Ejemplo visual de cómo se vería tu estructura interna:

```
raíz-del-proyecto/  
└─ docs/  
    └─ _2526-pmdm/          <-- (Carpeta oculta del histórico pasado. Docusaurus la  
ignora)  
        └─ apuntes viejos...  
            └─ 2627-pmdm/      <-- (Nueva carpeta principal 100% visible para los  
alumnos actuales)  
                └─ 01-introduccion.md
```

⚠ ¡ACUÉRDATE DEL MENÚ SUPERIOR!

Si usas esta técnica y te creas una nueva carpeta maestro `2627-pmdm/`, los menús superiores de tu página web seguirán rotos intentando apuntar a la vieja.

Tienes que abrir tu `docusaurus.config.js` y editar el botón de la asignatura en `themeConfig > navbar > items` para que su enlace o `to` redirija correctamente hacia `/docs/2627-pmdm/...`.

Estrategia 2: Versionado Automático Global

Docusaurus tiene un sistema nativo integrado de "versionado" (pensado originalmente para ir guardando versiones de programas informáticos como v1.0, v2.0...) que nosotros podemos aprovechar perfectamente para guardar nuestro histórico de "Cursos Académicos" sin romper manuales antiguos.

Al usar este método, Docusaurus **congela** todo tu temario actual de golpe, guardándolo intacto como archivo histórico para los alumnos. Al mismo tiempo, te deja vía libre en tu carpeta base (`docs/`) para que rehagas apuntes para el nuevo curso.

Cómo congelar tus apuntes actuales

Es algo que normalmente harás solo una vez al año, en junio. Para crear esa copia estática irrompible del curso que acabas, abre la terminal y ejecuta este comando, sustituyendo 24-25 por tu curso escolar:

```
npm run docusaurus docs:version 24-25
```

Cuando lo lanzas, el motor de Docusaurus hace todo el trabajo oscuro por ti:

1. Crea una carpeta local de archivo llamada `versioned docs/version-24-25/`.

2. Clona enterita la carpeta `docs/` que usabas como temario y se la lleva a esa nueva ruta blindada.
3. El material original central se queda vacío a nivel visible, pero a partir de este instante, todo lo modificado en `docs/` lo tratará como el temario para la **nueva promoción del año que viene**.

Añade el menú temporal

Para que el alumno (especialmente los repetidores o los de convocatoria extraordinaria) puedan revisar sus temarios congelados, tienes que habilitar el seleccionador desplegable. Configúralo en la ruta `themeConfig > navbar > items` de tu config:

`docusaurus.config.js`

```
items: [  
  {  
    type: 'docsVersionDropdown',  
    position: 'left',  
  },  
],
```

Evitar el cartel de "Versión no publicada"

Por pura inercia en la mentalidad informática, Docusaurus asume que la versión que has congelado es tu versión final maestra, y que lo que guardas en tu carpeta original `docs/` es un simple borrador. Por eso, mostrará por sistema un feo cartel (*banner*) amarillo avisando al alumno de que el temario de "este año" es falso e invitándole a ir al del año pasado.

Para invertir la mentalidad y asegurar que tu curso en vigor es el correcto, añade este bloque a la zona de plugins `presets` de tu `docusaurus.config.js`:

`docusaurus.config.js`

```
presets: [  
  [  
    'classic',  
    ({  
      docs: {  
        sidebarPath: './sidebars.js',  
  
        // --- AÑADE ESTE ÚNICO BLOQUE ---
```

```
lastVersion: 'current', // Exige que la carpeta principal en vigor sea la
primera
versions: {
  current: { banner: 'none' }, // Apaga la advertencia de "No publicado" a
tus nuevos alumnos
  '24-25': { banner: 'unmaintained' }, // Enciende el aviso a quienes miren
el histórico
},
// -----
```

DOCENTES CON MÚLTIPLES ASIGNATURAS

El comando básico automático le saca una fotografía a la carpeta `docs/` **entera**. Si almacenas apuntes de varias asignaturas distintas (ej: Docusaurus y PMDM) y lanzas el comando, versionarás todas simultáneamente en el mismo bloque para un curso concreto. Si quieres poder versionar primero PMDM y luego otra materia de forma independiente, investiga la técnica orientada al **"Multi-instance"** de la documentación oficial.

Exportación Offline a PDF

Tarde o temprano, ya sea el primer día de clase o en vísperas del examen, un alumno te hará la pregunta del millón: "Profe, ¿tienes todos estos apuntes en PDF para poder imprimirlos o usarlos sin internet?".

Aunque nuestra filosofía creando este ecosistema web (*Docs-as-Code*) es que el temario sea una experiencia interactiva (con vídeos, buscador y código fácil de copiar), hay situaciones donde generar un archivo físico tradicional es necesario.

Para lograrlo nos apoyaremos en complementos (*plugins*) y simuladores de navegador creados por la comunidad, dado que el código puro de Docusaurus no incluye un botón de fábrica para "Exportar a PDF".

❗ EL ECOSISTEMA DE LA COMUNIDAD

Igual que añadiremos el generador de PDF, Docusaurus cuenta con un enorme catálogo de herramientas gratuitas creadas por otros desarrolladores de todo el mundo. Puedes explorar la lista completa de extensiones avaladas en su página oficial de [Soporte y Comunidad](#).

Ejecutar el generador automático (`docs-to-pdf`)

El método más robusto para convertir tu curso en un libro es utilizar un "rastreador web". Básicamente, un mini programa oculto entra en tu página local, clics automáticamente en "Siguiente Tema" y va guardando capturas en PDF de cada apartado hasta fusionarlas en un documento maestro.

Vamos a aprovechar el paquete: `docs-to-pdf` de Jean Humann. Su gran ventaja es que no necesita instalarse internamente como plugin en nuestro archivo `docusaurus.config.js`, sino que se ejecuta temporalmente en la terminal evitando romper configuraciones.

Sigue estos **dos pasos principales**:

- 1. Asegúrate de que la web esté encendida:** El rastreador robot necesita poder "leer" tu web en vivo. Mantén tu servidor arrancado con normalidad (`npm run start`) para que la web cargue correctamente (por ejemplo, en `http://localhost:3000`).
- 2. Abre otra consola y dispara el PDF:** Una vez que sepas que la web está encendida, abre una **nueva pestaña** en tu terminal y escribe el siguiente comando `npx`. (Solo tienes que sustituir la URL larga

por el link de la primera página introductoria de tus apuntes reales):

```
npx docs-to-pdf docusaurus --  
initialDocURLs="http://localhost:3000/DevTacoraTeachers/docs/category/sesi%C3%B3n-  
3-despliegue-b%C3%BAsqueda-y-escalabilidad"
```

(Nota técnica: El comando `npx` evitará fallos de NPM porque se limitará a descargar momentáneamente la librería en caché y usarla solo para esta tarea).

¿Dónde encuentro el archivo?

Cuando el proceso finalice (pudiendo tardarse algo de tiempo si el módulo de apuntes es pesado), el sistema creará un colosal archivo físico en PDF en la propia raíz de tu base de directorios (cerca de tu `package.json`).

Estará compuesto por todas y cada una de tus secciones sin saltos visuales ni rupturas en los menús, ideal para ser distribuido a tus clases en la Moodle o imprimirse enteramente a petición de jefatura.

TIP EXTRA: LA IMPRESIÓN INSTANTÁNEA

Hay un detalle que salvará la vida al 90% de vuestros estudiantes: Si lo único que necesitan es imprimir **un solo tema** concreto de esos apuntes para repasar en el autobús, infórmalos de que pueden pulsar **Ctrl + P** (o Archivo -> Imprimir) directamente desde cualquier navegador web moderno (Chrome, Edge, Firefox...).

Docusaurus porta de fábrica un código de diseño oculto "solo para impresión" que hace que al imprimir desaparezcan mágicamente los menús laterales o el buscador superior, generando un folio blanco puramente de notas y texto para el alumno.